Justificación de la optimización del controlador difuso de semáforos

Cómo se hizo y por qué se eligió el controlador FLC-O

Documento de apoyo para la revisión del artículo «Mejora de un controlador de semáforos de lógica difusa para una intersección aislada: optimización del controlador de Pappis y Mamdani (1977)»

Raul Nolberto Quispe Saya y Kevin Luis Valero Ccacya
Escuela Profesional de Ingeniería Electrónica, Universidad Nacional de San Agustín de Arequipa

1. Resumen

Este documento responde a dos preguntas que plantea el artículo: ¿cómo se optimizó el controlador? y ¿por qué se eligió, entre varias alternativas, el controlador FLC-O? Todas las cifras salen de los resultados guardados en el repositorio (`resultados/opt/`) y se regeneran con `codigo/gen_justificacion.py` y `codigo/gen_tablas_opt.py`.

En una frase. Se buscó un controlador difuso mejor que el de Pappis y Mamdani (1977) con un procedimiento que no deja lugar a trampas de medición: un simulador exacto, un objetivo que cuenta toda la espera, tres grupos de escenarios separados (entrenar, elegir y probar) y rivales afinados con el mismo presupuesto. Se probaron cinco familias de candidatos y se eligió FLC-O porque fue el mejor en el grupo reservado para elegir, rindió igual o mejor que una red neuronal tres veces más grande conservando 27 reglas legibles, y su ventaja se mantuvo en cuatro conjuntos de prueba que nunca se usaron para entrenar.

Qué se hizo, en cinco pasos.

1. Se corrigió la medida del retardo para que cuente toda la espera hasta el final de la simulación (Sec. 3.1).
2. Se reescribió el simulador para evaluar cientos de escenarios a la vez, con resultados idénticos al simulador de referencia (Sec. 3.2).
3. Se definió un objetivo $J(\theta)$ y se optimizaron los parámetros con CMA-ES, un algoritmo evolutivo que no necesita derivadas (Sec. 3.3–3.4).
4. Se separaron los escenarios en entrenamiento, validación y prueba, y se afinaron con el mismo presupuesto los rivales (Sec. 3.5–3.6).
5. Se compararon cinco familias de candidatos y se eligió una con la validación, no con la prueba (Sec. 4–5).

Resultado. Retardo medio en s/veh con la vista adelante de 10 s que supone el modelo original:

Conjunto de prueba	Pappis	Actuado af.	FLC-O	FLC-O vs Pappis	FLC-O vs actuado
Oficial (16 pares × 20 réplicas)	10.95	9.63	7.91	−27.8 %	−17.8 %
64 caudales no vistos	10.40	9.17	7.40	−28.8 %	−19.3 %
Demanda variable	8.65	8.07	6.07	−29.8 %	−24.7 %
Cerca de la saturación	105.87	24.13	24.29	−77.1 %	empate (−0.7 %)

Fuente: `resultados/opt/evaluacion_final.json`; tabla del artículo generada por `codigo/gen_tablas_opt.py`.

Lo que no se afirma. (i) La mejora depende de la vista adelante de 10 s; sin ella el mejor controlador aprendido supera al actuado afinado solo en 1.7 %. (ii) Cerca de la saturación hay empate con el actuado afinado, no mejora. (iii) Todo es simulación; no hay datos de campo. Estos límites se detallan en la Sec. 9.

2. Punto de partida: qué se quería mejorar

El controlador de Pappis y Mamdani [1] gobierna una intersección aislada de dos calles unidireccionales. Interviene a los 7, 17, 27, 37 y 47 s de verde y, con 25 reglas, decide cuántos segundos (de 1 a 10) extender el verde o si cambiar de fase. La réplica previa de los autores lo reprodujo y dejó tres hechos que motivaron este trabajo:

1. Mejora al ciclo fijo de Webster en 16 de 16 combinaciones de caudal, como publicó el original.
2. Frente a un controlador *actuado* por detector, que el original no incluye, no es mejor en promedio (10.95 contra 10.89 s/veh en el conjunto oficial).
3. Sus reglas se ajustaron por prueba y error en 1977; no hay razón para pensar que sean las mejores.

La pregunta de investigación fue entonces: *¿cuánto puede mejorarse ese controlador si se optimizan sus parámetros y su estructura con un procedimiento reproducible, y contra qué referencias justas hay que medirlo?* El criterio de éxito adoptado fue ser mejor que el original *y* que un rival fuerte (el actuado afinado), en escenarios que no se usaron para entrenar.

3. Cómo se hizo la optimización

3.1. La medida de desempeño: el retardo que cuenta toda la espera

La medida es el retardo medio por vehículo

$$d_m = \frac{1}{N} \sum_{n=0}^{H-1} \big(Q_{1,n} + Q_{2,n}\big), \tag{1}$$

con N los vehículos llegados, H el horizonte (7200 s en las pruebas) y $Q_{i,n}$ la cola del brazo i en el segundo n. *Origen:* cada vehículo en cola acumula un segundo de espera por segundo, así que sumar las colas segundo a segundo suma las esperas (definición propia; por la ley de Little [2], $L = \lambda W$, es equivalente a la espera media por vehículo cuando se la mide hasta que sale). El original solo cuenta a los vehículos que ya salieron; la diferencia es la espera de quienes siguen en la cola al terminar. Importa cuando el controlador deja crecer la cola:

	Oficial		Saturado	
	solo servidos	con residuo	solo servidos	con residuo
Pappis y Mamdani	10.93	10.95	99.46	105.87
FLC-O	7.90	7.91	24.22	24.29

Fuente: `resultados/opt/evaluacion_final.json`, campos `delay` y `delay_servidos`.

En el conjunto oficial la diferencia es mínima, pero cerca de la saturación la convención del original oculta el 6.1 % del retardo de Pappis y Mamdani. Un objetivo que ignorara esa espera premiaría a un controlador que abandona colas. Por eso el artículo usa (1) y reporta aparte, en `resultados/opt/evaluacion_final.json`, la convención del original.

3.2. El simulador vectorizado

Optimizar exige evaluar un controlador muchas veces. Con $\lambda = 15$ candidatos por generación, 150 generaciones y 64 escenarios, cada entrenamiento de FLC-O hace 144 000 simulaciones de 3600 s. Con el simulador de referencia (unos 230 ms por escenario de 7200 s, medido con el controlador de Pappis y Mamdani) eso son del orden de 4.6 h en un núcleo, y se necesitan varios entrenamientos. Se

reescribió el simulador para simular n_e escenarios a la vez con arreglos de NumPy: unos 10 ms por escenario, unas veinte veces menos. Lo que hace válido el cambio es que la equivalencia es *exacta*: en 8 escenarios de 2400 s, con seis controladores y las dos convenciones de retardo, la diferencia es nula (`test_sim_batch.py`). El simulador interactivo con calle y autos usa la misma lógica y se comprueba contra este (`test_sim_flco.py`).

3.3. El objetivo

Para un vector de parámetros θ se minimiza

$$J(\theta) = \frac{1}{N_e} \sum_{i=1}^{N_e} \frac{d_m^{(i)}(\theta)}{d_m^{(i)}(\text{act})}, \qquad N_e = 64, \tag{2}$$

la media, sobre 64 escenarios, del retardo del controlador dividido por el del actuado afinado en el *mismo* escenario (definición propia). Tres razones: (a) $J < 1$ significa «mejor que un rival fuerte», no solo «mejor que nada»; (b) todos los niveles de demanda pesan igual, porque sin dividir, los escenarios de mucho tráfico, con retardos grandes, dominarían el promedio y el controlador aprendería solo a manejarlos; (c) las llegadas de cada escenario son las mismas para todos los candidatos, de modo que la comparación entre dos candidatos no depende de la suerte de las llegadas.

3.4. El optimizador: CMA-ES

Los parámetros se optimizan con CMA-ES [3, 4], una estrategia evolutiva que mantiene una distribución normal sobre los parámetros, sortea candidatos, evalúa cada uno con (2) y usa los mejores para actualizar la media y la covarianza de la distribución. Se eligió porque el objetivo es una simulación con eventos discretos, sin derivadas utilizables, y porque con 30 a 121 parámetros es el rango en que este método se usa habitualmente; tiene pocos parámetros propios que ajustar. Configuración común: población $\lambda = 4 + \lfloor 3 \ln n \rfloor$ ($\lambda = 15$ para $n = 42$), $\mu = \lfloor \lambda/2 \rfloor = 7$ padres, paso inicial $\sigma_0 = 0.3$, 150 generaciones (de 80 a 200 según el modelo), evaluación en paralelo con 4 procesos y puntos de control cada 10 generaciones. Entrenar FLC-O toma unos 15 minutos. La implementación es propia (`codigo/opt_es.py`) y se verifica con funciones de prueba conocidas (esfera y Rosenbrock, `test_opt.py`). No se comparó con otros optimizadores; es una decisión de diseño, no un resultado.

3.5. Tres grupos de escenarios que nunca se mezclan

Para que el resultado no sea un ajuste a los datos, cada decisión usa un grupo distinto:

Grupo	Contenido	Semillas	Para qué se usa
Entrenamiento	64 pares de caudales sorteados (cada brazo $\geq$ 180, suma de 540 a 3300 veh/h), 3600 s	100000 a 100063	Calcular J y entrenar
Validación	64 pares distintos, 7200 s	200000 a 200063	**Elegir** entre candidatos, el verde mínimo y la semilla
Prueba oficial	Los 16 pares de la Tabla VIII del original $\times$ 20 réplicas, 7200 s	0 a 19	Resultado principal
Prueba: no vistos	64 pares sorteados con otras semillas, 7200 s	500000 en adelante	Que no dependa de los 16 pares
Prueba: variable	64 escenarios con demanda que cambia cada 600 s (factor de 0.4 a 1.6), 7200 s	–	Demanda no estacionaria
Prueba: saturado	32 pares con suma de 3000 a 3300 veh/h, 7200 s	400000 en adelante	Cerca de la capacidad

Fuente: `codigo/opt_entrenar.py` y `codigo/opt_evaluar.py`; hiperparámetros en el suplemento del artículo.

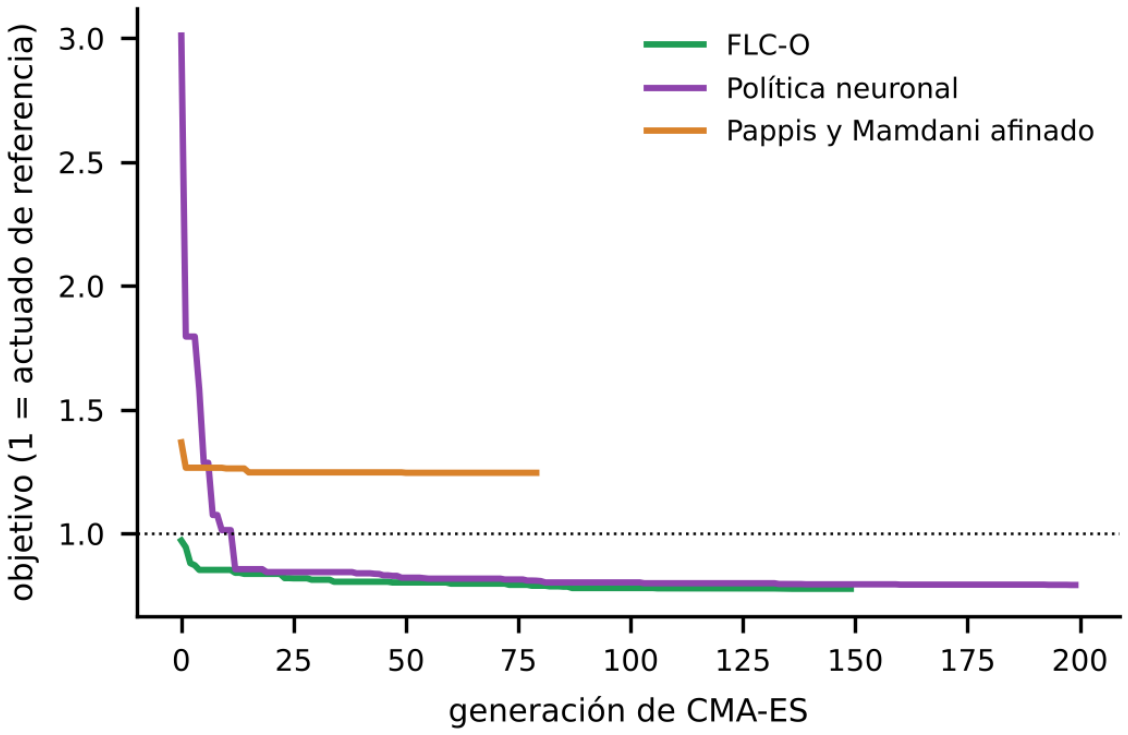


Fig. 1: Mejor valor del objetivo $J(\theta)$ por generación para los modelos finales (1: igual al actuado afinado de referencia; menos es mejor).

Ningún conjunto de entrenamiento incluye los 16 pares de la Tabla VIII. Las elecciones de diseño (candidato, verde mínimo y semilla) se hicieron con la validación y no con la prueba, y los modelos finales no se reentrenaron a partir de los resultados de prueba. Los conjuntos de prueba se evaluaron con todos los candidatos, no solo con el elegido.

3.6. Rivales justos

Una mejora solo vale frente a un rival serio. Por eso, además del ciclo fijo de Webster y del actuado de la réplica, se afinaron dos controladores de la literatura con los mismos escenarios de entrenamiento, el mismo rango de verde mínimo (3 a 7 s) y el mismo verde máximo (hasta 90 s) que los controladores aprendidos:

- **Actuado afinado:** se probaron 45 configuraciones (hueco de 1 a 3 s, verde mínimo de 3 a 7 s, máximo de 57, 80 y 90 s). Resultó hueco de 1 s, mínimo de 3 s y máximo de 90 s, con un J de 0.823 respecto del actuado original: ya es un 17.7 % mejor que el original. Es el rival fuerte del artículo.
- **Presión máxima** [5]: afinada igual, da 17.44 s/veh. En este modelo, con dos fases y tiempo perdido, rinde peor que el actuado; se incluye como referencia de la literatura, no como rival fuerte.
- **Cota por programación dinámica** [6]: la política óptima para caudales *conocidos* y sin ver llegadas futuras. Da 9.35 s/veh y se valida contra la simulación de su propia política (diferencia máxima 0.7 %). Sirve como comprobación de coherencia (Sec. 7).

Un error que se detectó y corrigió. La primera rejilla del actuado exigía un verde mínimo de 5 s o más, lo que lo debilitaba frente a controladores que podían usar 3 s y favorecía al propuesto. Se amplió el rango a 3–7 s y el actuado mejoró un 5 % en el conjunto oficial; todas las cifras del artículo usan esta versión justa. Lo mismo se hizo con la presión máxima. Las rejillas anteriores se conservan en `resultados/opt/actuado_afinado_v1.json` y `resultados/opt/maxpressure_afinado_v1.json`.

Tabla 1: Candidatos entrenados con las mismas condiciones: J de entrenamiento (3600 s, 64 escenarios) y de validación (7200 s, 64 escenarios distintos; 1 es igual al actuado afinado, menos es mejor) y retardo medio en los cuatro conjuntos de prueba (s/veh)

Candidato	Par.	Vista	J entr.	J val.	Oficial	No vistos	Variable	Saturado
Actuado afinado (referencia)	3	no	–	1	9.63	9.17	8.07	24.13
Pappis y Mamdani (1977)	0	10 s	–	–	10.95	10.40	8.65	105.87
Pappis y Mamdani afinado (25 reglas)	30	10 s	1.246	1.420	9.67	9.30	8.06	58.11
Difuso 27 reglas, horizontes fijos	36	10 s	0.826	0.837	8.44	7.88	6.56	24.80
Difuso 81 reglas con hueco	93	10 s	0.827	0.838	8.47	7.93	6.55	25.00
FLC-O, $g_{\text{mín}} = 7\,\text{s}$	42	10 s	0.860	0.899	8.60	8.14	6.85	25.15
FLC-O, $g_{\text{mín}} = 5\,\text{s}$	42	10 s	0.815	0.834	8.21	7.75	6.41	25.16
FLC-O, $g_{\text{mín}} = 4\,\text{s}$	42	10 s	0.795	0.807	8.04	7.58	6.20	24.38
FLC-O, $g_{\text{mín}} = 3\,\text{s}$, semilla 0	42	10 s	0.794	0.805	8.00	7.51	6.21	24.79
FLC-O, $g_{\text{mín}} = 3\,\text{s}$, semilla 1	42	10 s	0.788	0.807	8.04	7.49	6.24	24.63
FLC-O, $g_{\text{mín}} = 3\,\text{s}$, semilla 2 (elegido)	42	10 s	0.777	0.800	7.91	7.40	6.07	24.29
Política neuronal	121	10 s	0.793	0.809	8.09	7.53	6.24	25.10
Difuso 27 reglas, causal	36	no	0.982	1.065	9.97	9.37	8.41	25.74
Política lineal causal	16	no	0.949	0.992	9.53	8.97	8.10	25.11
Política neuronal causal	121	no	0.942	0.982	9.46	8.95	8.05	24.29

Fuente: `resultados/opt/evaluacion_final.json` (prueba), `codigo/opt_validar.py` (validación) y campo `fitness_train` de cada modelo de `resultados/opt/` (entrenamiento); tabla generada por `codigo/gen_justificacion.py`.

4. Qué candidatos se probaron

Se entrenaron, con las mismas condiciones, cinco familias. Cada una responde a una pregunta distinta:

A. Pappis y Mamdani afinado (30 parámetros). *¿Basta con ajustar el controlador de 1977?* Conserva sus 25 reglas y su estructura y afina el umbral de decisión, una escala y un desplazamiento de las entradas A y Q, y un desplazamiento del conjunto de tiempo de cada regla. Con los valores del original reproduce exactamente al controlador replicado. Resultado: 9.67 s/veh (una mejora de 11.7 % respecto de Pappis y Mamdani) y, cerca de la saturación, 58.11 s/veh, más del doble que el actuado afinado: con la estructura de 1977 conservada, ajustar ayuda pero no es suficiente.

B. Difuso nuevo de 27 reglas con horizontes fijos (36 parámetros). *¿Ayuda cambiar la estructura: decidir cada segundo, con menos reglas?* Tres variables (demanda en verde G, presión en rojo R, verde transcurrido V) con tres conjuntos cada una. Resultado: 8.44 s/veh; ya supera con holgura a la familia A.

C. Difuso de 81 reglas con el hueco desde la última llegada (93 parámetros). *¿Más reglas y más información mejoran?* Agrega como cuarta variable el tiempo desde la última llegada al brazo con verde, la señal que usa el actuado. Resultado: 8.47 s/veh, sin mejora; la información adicional no aporta y los parámetros se triplican.

D. FLC-O (42 parámetros). *¿Se puede aprender también cuánto mirar hacia adelante?* Es la familia B con una modificación: las llegadas previstas que alimentan G y R son una combinación lineal, con pesos no negativos *aprendidos*, de las llegadas de los próximos 2, 5 y 10 s (verde) y 3, 6 y 10 s (rojo). Resultado: 8.00 s/veh con la misma semilla que B, y 7.91 con la semilla elegida.

E. Política neuronal (121 parámetros). *¿Cuánto se pierde por exigir reglas legibles?* Una red de una capa oculta con 8 neuronas que recibe 12 características y decide cada segundo. No es un controlador propuesto, sino una referencia de lo que logra un aprendizaje sin estructura con la misma información, también con sus variantes sin vista adelante (causales).

5. Por qué se eligió FLC-O

5.1. Los criterios

Los dos primeros decidieron la elección y se aplicaron con la validación; los otros tres son comprobaciones con los conjuntos de prueba. En orden: (1) mejor J en validación; (2) a igual rendimiento, preferir el modelo con menos parámetros y con reglas que puedan leerse; (3) que la ventaja se mantenga en datos no vistos y con demanda variable; (4) que no se degrade cerca de la saturación; (5) que no dependa de la semilla de entrenamiento.

5.2. Las razones

1. **Fue el mejor en el grupo reservado para elegir.** En validación, FLC-O con $g_{\text{mín}} = 3\,\text{s}$ y la semilla 2 tiene $J = 0.800$, contra 0.809 de la red neuronal, 0.837 y 0.838 de las otras dos familias difusas y 1.420 de Pappis y Mamdani afinado (Tabla 1). La elección se hizo con esta columna, no con la de prueba.
2. **La legibilidad no cuesta rendimiento.** FLC-O no es peor que una red con 121 parámetros: 7.91 contra 8.09 s/veh en el conjunto oficial. Comparados por par de caudales con Wilcoxon pareado y corrección de Holm, FLC-O es significativamente mejor en 11 de 16 pares y en ninguno es peor (diferencia media de 2.5 %). El margen es pequeño y hay una asimetría que conviene decir: FLC-O es la mejor de tres semillas y la red es un solo entrenamiento; con el promedio de las tres semillas (7.98 s/veh) la ventaja sobre la red sigue, pero baja a 1.3 %. No se afirma que FLC-O sea mejor que cualquier red posible (Sec. 9); lo que sí muestra es que exigir 27 reglas legibles no obliga a rendir menos.
3. **Aprender la vista adelante aporta.** Con horizontes fijos, el mismo difuso de 27 reglas da 8.44 s/veh; con los pesos aprendidos, 8.00 con la misma semilla.
4. **Más reglas no ayudan.** La variante de 81 reglas da 8.47 s/veh. Con 27 reglas, la base se puede leer entera en una tabla (Tabla 2).
5. **Generaliza.** El retardo de FLC-O es 0.822 veces el del actuado afinado en el conjunto oficial, 0.807 en los caudales no vistos y 0.753 con demanda variable. La brecha entre J de entrenamiento (0.777) y de validación (0.800) es de 2.9 %, pequeña para un modelo de 42 parámetros.
6. **No se degrada cerca de la saturación.** 24.29 s/veh frente a 24.13 del actuado afinado: empate. Pappis y Mamdani da 105.87 y su versión afinada 58.11.
7. **No depende de una semilla afortunada.** Se entrenó con tres semillas: en el conjunto oficial dan 8.00 (semilla 0), 8.04 (semilla 1) y 7.91 s/veh (semilla 2, la elegida). Si en lugar de la elegida por validación se tomara el promedio de las tres (7.98 s/veh), la reducción frente a Pappis y Mamdani sería de 27.1 %, y con la peor, de 26.6 %; la conclusión no cambia.

5.3. Cómo se eligió la configuración final

- **Verde mínimo:** se entrenó FLC-O con 3, 4, 5 y 7 s y se eligió con la validación: $J = 0.805$ (3 s), 0.807 (4 s), 0.834 (5 s) y 0.899 (7 s). Es el factor que más pesa. Si la norma local exige más de 3 s, con 5 s se conserva casi toda la mejora (8.21 s/veh, 25.1 % menos que Pappis y Mamdani).
- **Semilla:** de las tres semillas con 3 s ($J = 0.805$, 0.807 y 0.800) se tomó la de menor J en validación. Esa semilla resultó también la de menor retardo en la prueba, pero la elección no se hizo mirándola.
- **Verde máximo de 90 s:** permite a los controladores aprendidos no depender del tope de 57 s de 1977, que limita a Pappis y Mamdani cerca de la saturación; el actuado afinado recibió el mismo tope.

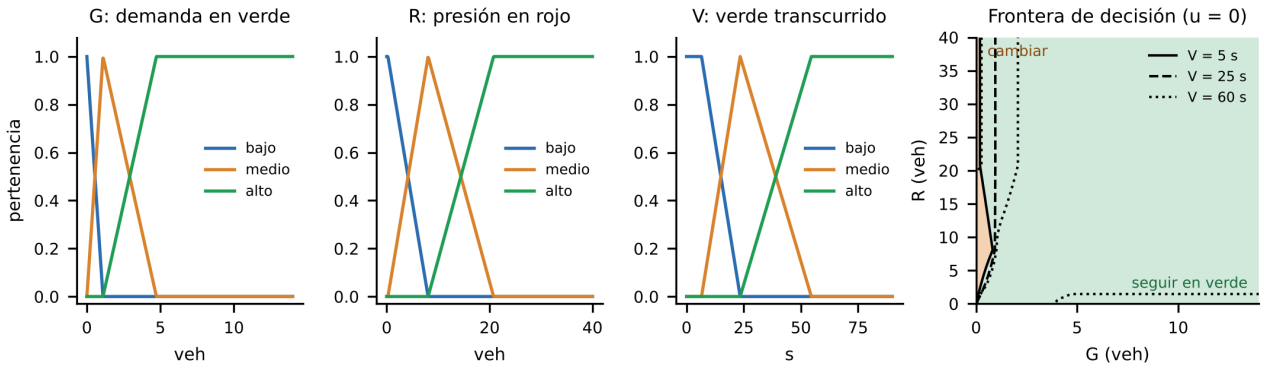


Fig. 2: Funciones de pertenencia aprendidas de FLC-O para G, R y V y frontera de decisión $u = 0$ en el plano (G, R) para tres valores de V.

6. Qué aprendió FLC-O (lectura del modelo)

Las 27 reglas tienen la forma «si G es $\mathcal{G}_i$ y R es $\mathcal{R}_j$ y V es $\mathcal{V}_k$ entonces c_{ijk}»; la salida es

$$u = \frac{\sum_{ijk} w_{ijk}\, c_{ijk}}{\sum_{ijk} w_{ijk}}, \qquad \text{cambiar de fase} \iff u > 0, \tag{3}$$

con w_{ijk} el producto de las tres pertenencias. *Origen:* es el centro de gravedad de consecuentes de un solo punto, un sistema de Mamdani con salidas *singleton* equivalente a uno de Takagi y Sugeno de orden cero [7]. Se eligió esta forma porque da una salida suave, que facilita el trabajo de CMA-ES, y a la vez cada regla dice con un número si conviene cambiar ($c > 0$) o seguir ($c < 0$).

Las entradas son $G = Q_g + \sum_k \omega_k W_g(l_k)$ y $R = Q_r + \sum_k \rho_k W_r(l'_k)$, con Q_g y Q_r las colas de los brazos en verde y en rojo, $W(l)$ las llegadas previstas en los l segundos siguientes ($l = 2, 5, 10$ en verde y $l' = 3, 6, 10$ en rojo) y $\omega_k, \rho_k \geq 0$ pesos que aprende el entrenamiento; V son los segundos de verde transcurridos. Los pesos aprendidos son $\omega = (0.88, 0.21, 0.03)$ y $\rho = (0.22, 0.88, 0.49)$. Como cada ventana contiene a la anterior, el peso efectivo de un vehículo según su segundo de llegada es:

	primeros segundos	segundos intermedios	segundos lejanos
Verde (G)	s 1–2: 1.12	s 3–5: 0.24	s 6–10: 0.03
Rojo (R)	s 1–3: 1.58	s 4–6: 1.37	s 7–10: 0.49

Fuente: cálculo sobre los pesos del modelo `resultados/opt/flcopt_g3_s2.json`, hecho por `codigo/gen_justificacion.py`.

Interpretación (una lectura del resultado, no una demostración): para decidir si seguir en verde importan sobre todo los vehículos que llegarán en los próximos 2 s, que son los que aún aprovecharían el verde; para estimar la presión en rojo importan los que llegarán en los próximos 6 s, un horizonte parecido a los 5 s de tiempo perdido de cada cambio de fase. Los conjuntos aprendidos tienen sus nudos en $G = (0.00, 1.10, 4.73)$, $R = (0.24, 8.05, 20.78)$ y $V = (6.56, 23.46, 54.57)$ s. De los 27 consecuentes, 9 empujan a cambiar y 18 a seguir, con valores de -3.32 a 2.37. El patrón es el esperable: se cambia cuando ya queda poca demanda en verde y hay presión en rojo (fila G bajo, R medio: de $+1.45$ a $+2.37$), se sigue cuando la demanda en verde es media o alta (por ejemplo G alto, R medio: de -3.32 a -1.81), y también cuando ambos brazos están muy cargados, para no malgastar tiempo perdido.

7. Evidencias de que la mejora es real

1. **Datos separados.** Entrenar, elegir y probar usan escenarios y semillas distintos (Sec. 3.5).
2. **Significancia estadística por par.** En el conjunto oficial, FLC-O es significativamente mejor que Pappis y Mamdani en 16 de 16 pares y que el actuado afinado en 16, con corrección de Holm y sin ningún par peor (Wilcoxon pareado sobre 20 réplicas con números aleatorios comunes).

Tabla 2: Base de reglas de FLC-O: consecuente numérico c_{ijk} de cada regla (positivo: cambiar la fase; negativo: seguir en verde) según la demanda en verde G, la presión en rojo R y el verde transcurrido V. Nudos (k_1, k_2, k_3): G (0.0, 1.1, 4.7), R (0.2, 8.0, 20.8), V (6.6, 23.5, 54.6). Pesos aprendidos de la vista adelante: verde (0.88, 0.21, 0.03) para 2, 5 y 10 s; rojo (0.22, 0.88, 0.49) para 3, 6 y 10 s.

G	R	V bajo	V medio	V alto
bajo	bajo	+0.06	+0.04	−0.16
bajo	medio	+1.52	+1.45	+2.37
bajo	alto	+0.34	+2.18	−0.19
medio	bajo	−2.75	−1.06	−1.09
medio	medio	−0.51	−0.27	−0.20
medio	alto	−1.61	−0.36	+0.60
alto	bajo	−0.52	−1.34	+0.33
alto	medio	−3.32	−2.08	−1.81
alto	alto	−2.04	−0.03	−1.62

Fuente: modelo entrenado flcopt_g3_s2 (resultados/opt/flcopt_g3_s2.json).

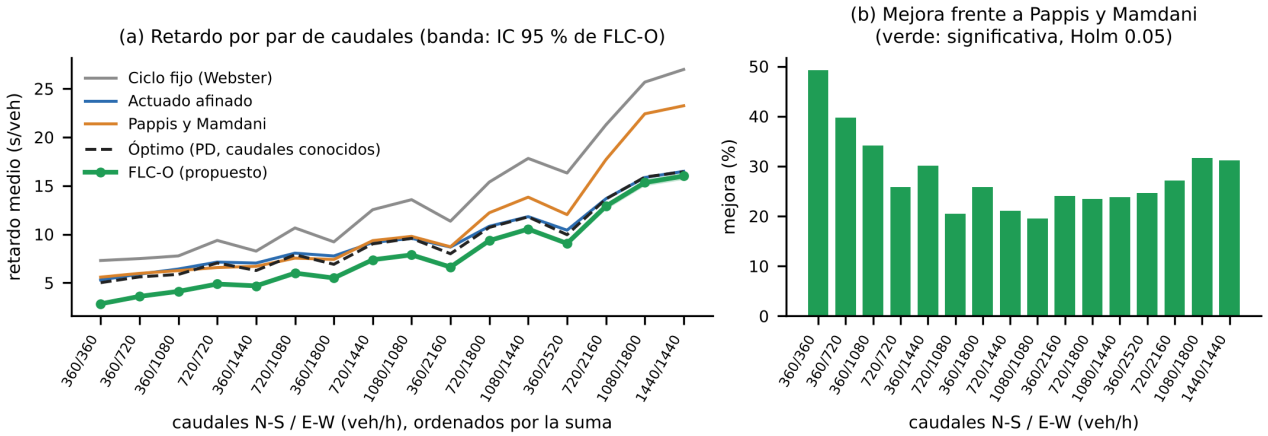


Fig. 3: (a) Retardo medio por par de caudales de la Tabla VIII (7200 s, 20 réplicas); la banda es el intervalo de confianza de 95 % de FLC-O. (b) Mejora de FLC-O frente a Pappis y Mamdani en cada par; en verde, las significativas con la corrección de Holm.

3. **Datos no vistos y demanda variable.** La reducción se mantiene: 28.8 % y 19.3 % con 64 caudales no vistos; 29.8 % y 24.7 % con demanda variable.
4. **Simulador exacto.** El simulador rápido da, escenario por escenario, el mismo retardo que el de referencia, así que ninguna ventaja viene de una simplificación del simulador.
5. **Comprobación de coherencia con la cota.** La cota por programación dinámica (9.35 s/veh) es el mejor retardo posible para quien conoce los caudales pero no ve el futuro. Todos los controladores *sin* vista adelante quedan por encima: la política neuronal causal da 9.46, la lineal causal 9.53, el actuado afinado 9.63 y el difuso causal 9.97. Es coherente con un entrenamiento que no explota atajos del simulador (no lo demuestra, pero un controlador sin vista adelante por debajo de la cota habría sido una señal de alarma). FLC-O queda por debajo de la cota porque sí ve las llegadas de los próximos segundos, no porque la cota falle.
6. **Rivales afinados con el mismo presupuesto**, y un error de rejilla que se corrigió en contra del propuesto (Sec. 3.6).
7. **Todo es reproducible** desde el repositorio (Sec. 11).

8. Qué se corrigió y se descartó en el camino

Se deja constancia porque ayuda a juzgar el procedimiento:

- **Rejilla del actuado** que favorecía al propuesto: corregida (Sec. 3.6).
- **La red neuronal ya no se presenta como cota.** Una versión anterior del texto la llamaba «cota»; FLC-O la iguala o supera, así que solo es una referencia empírica.
- **Saturación.** Al principio los controladores entrenados con caudales de hasta 3000 veh/h se degradaban cerca de la capacidad; entrenar con hasta 3300 lo evitó. El resultado final es empate con el actuado afinado, y así se declara; no se presenta como mejora.
- **Verde mínimo de 7 s.** Los primeros entrenamientos lo usaban y limitaban a los modelos (con 7 s, FLC-O da 8.60 s/veh); se estudió el rango de 3 a 7 s con la validación.
- **El hueco desde la última llegada** no ayudó (familia C), pese a ser la señal del actuado.
- **Modelos exploratorios** con otras condiciones se conservan en `resultados/opt/previos/` por transparencia; no se usan en el artículo ni son comparables.

9. Limitaciones

1. **Vista adelante.** FLC-O usa las llegadas de los próximos 10 s, un supuesto del original (equivale a detectores aguas arriba sin error). Sin ella, FLC-O causal da 9.97 s/veh, peor que el actuado afinado (9.63); la mejor variante causal, la red neuronal, lo supera solo en 1.7 %. Con vista adelante, FLC-O tiene un retardo 16.4 % menor que el de la mejor variante sin ella (9.46 s/veh). Es la limitación más importante: en la práctica los detectores tienen error y alcance limitado, y no se midió la robustez ante eso.
2. **Saturación.** Empate con el actuado afinado, no mejora.
3. **Verde mínimo de 3 s.** Puede no ser admisible por seguridad o norma; con 5 s el resultado es 8.21 s/veh.
4. **Simulación.** Un cruce aislado, llegadas de Bernoulli, flujo de saturación fijo, sin giros, peatones ni redes. Los parámetros se entrenan y se evalúan en el mismo simulador, aunque con caudales y semillas distintos. No hay datos de campo. Los modelos exploratorios (Sec. 8) guiaron el diseño de las familias de candidatos; por eso las decisiones finales se tomaron con la validación y se reportan todas las variantes, incluidas las peores.
5. **Alcance de las comparaciones.** El actuado afinado y la presión máxima se afinaron con rejillas finitas; no hay garantía de que sean los mejores de su clase. La red neuronal es de una arquitectura y un entrenamiento; otra podría superarla. No se comparó CMA-ES con otros optimizadores.
6. **Medida.** El retardo corregido no es el que imprimía el original, así que las cifras no son comparables con las de su Tabla VIII.

10. Preguntas que podrían hacerse

¿Por qué no usar directamente la red neuronal? Rinde igual o peor (8.09 contra 7.91 s/veh; FLC-O es significativamente mejor en 11 de 16 pares), tiene tres veces más parámetros y no se puede leer ni auditar. Para un semáforo, que alguien pueda revisar por qué cambia de fase es una ventaja práctica.

¿Por qué CMA-ES y no un algoritmo genético u otro? Porque el objetivo es una simulación sin derivadas y CMA-ES es un método estándar para ese tipo de problema con decenas de parámetros [4]. No se comparó con otros; no se afirma que sea el mejor, solo que funciona y converge (Fig. 1).

¿Cómo sabemos que no hay sobreajuste? Porque se eligió con la validación y se probó en otros cuatro conjuntos; la brecha entre entrenamiento y validación es de 2.9 %, y la ventaja se mantiene con caudales no vistos (0.807 veces el actuado afinado) y con demanda variable (0.753).

¿Por qué el retardo no coincide con la Tabla VIII del original? Porque se cuenta también la espera de los vehículos que siguen en cola al final. El original no lo hacía. Las cifras de Pappis y Mamdani del artículo salen de la réplica con esta medida, no de la tabla impresa.

¿Por qué el verde mínimo es de 3 s? Porque fue el mejor en validación (Sec. 5). Es una elección de simulación; si hace falta más, con 5 s la mejora frente a Pappis y Mamdani sigue siendo de 25.1 %.

¿Por qué el controlador ve el futuro? Porque el modelo original ya lo supone (las llegadas de los próximos 10 s). Se mantuvo para comparar en las mismas condiciones y se midió cuánto depende de ello, que es mucho.

¿Se puede usar en una intersección real? No se afirma. Faltan datos de campo, errores de detección y los demás elementos de una intersección real (Sec. 9).

¿Por qué 27 reglas? Porque con tres conjuntos por variable (3^3) la base es la más pequeña que distingue demanda baja, media y alta; con 81 reglas no hubo mejora (familia C).

11. Reproducción y dónde está cada cosa

Con Python 3.13 y NumPy 2.5, desde la carpeta del proyecto:

- `python codigo/ejecutar_pruebas.py`: todas las comprobaciones.
- `python codigo/opt_validar.py` seguido de los nombres de los modelos: ordena candidatos con la validación.
- `python codigo/opt_evaluacion_final.py`: los cuatro conjuntos de prueba y la estadística.
- `python codigo/gen_tablas_opt.py` y `python codigo/gen_justificacion.py`: las cifras y tablas del artículo y de este documento.

Qué	Dónde
Simulador vectorizado y su prueba de equivalencia	`codigo/sim_batch.py`, `codigo/test_sim_batch.py`
Controladores (Pappis y Mamdani afinado, difusos, política neuronal, referencias)	`codigo/opt_controladores.py`, `opt_flc.py`, `opt_flc2.py`, `opt_politicas.py`
Optimizador CMA-ES	`codigo/opt_es.py`
Cota por programación dinámica	`codigo/opt_mdp.py`, `resultados/opt/mdp_oficial.json`
Entrenamiento y escenarios	`codigo/opt_entrenar.py`, `opt_entrenar_modelo.py`
Modelos entrenados y el elegido	`resultados/opt/flcopt_g3_s2.json` (el elegido) y `resultados/opt/modelos_finales.json`
Evaluación final y estadística	`codigo/opt_evaluacion_final.py`, `resultados/opt/evaluacion_final.json`
Rivales afinados	`resultados/opt/actuado_afinado.json` y `resultados/opt/maxpressure_afinado.json`
Simulador interactivo con calle y autos	`codigo/simulador_gui.py`, `codigo/web_sim.py`
Artículo, suplemento y este documento	`articulo/main.tex`, `suplemento.tex`, `justificacion.tex`

Fuente: repositorio del proyecto, rama `main`.

Referencias

[1] C. P. Pappis and E. H. Mamdani, "A fuzzy logic controller for a traffic junction," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-7, no. 10, pp. 707–716, Oct. 1977.

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Oper. Res.*, vol. 9, no. 3, pp. 383–387, 1961, doi:10.1287/opre.9.3.383.

[3] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evol. Comput.*, vol. 9, no. 2, pp. 159–195, 2001.

[4] N. Hansen, "The CMA evolution strategy: A tutorial," arXiv:1604.00772, 2016.

[5] P. Varaiya, "Max pressure control of a network of signalized intersections," *Transp. Res. C, Emerg. Technol.*, vol. 36, pp. 177–195, 2013.

[6] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York, NY, USA: Wiley, 1994.

[7] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-15, no. 1, pp. 116–132, Jan. 1985, doi:10.1109/TSMC.1985.6313399.